



Computación Distribuida

Práctica 01 - Equipo 02

Integrantes:

Humberto Reyes, Julián Humberto

García Bollás, Emiliano

Aguilar Romero, Pamela

Barreto Velázquez Frank

...

PRACTICA 1: Preguntas de análisis

1. ¿Cómo se divide el trabajo entre los diferentes procesos?

El proceso coordinador (server) toma el límite superior N y el número de trabajadores especificado, y divide el intervalo global $[1, N]$ en subintervalos de tamaño similar. Cada proceso trabajador (worker) recibe un rango de números específico (por ejemplo, $[a, b]$) sobre el cual debe aplicar la criba de Eratóstenes de forma independiente.

2. ¿Cómo se comunican los trabajadores con el proceso coordinador?

Se comunican mediante el mecanismo nativo de paso de mensajes asíncrono que ofrece Elixir/BEAM. El coordinador inicia los procesos trabajadores pasándoles su propio ID de proceso (PID) o manteniéndolo como referencia, y cada trabajador envía una estructura de datos/tupla con sus resultados usando `send/2`. El coordinador los recolecta mediante un bloque `receive`.

3. ¿Qué sucede si uno de los trabajadores tarda más que los demás?

El proceso coordinador debe esperar a que todos los trabajadores terminen y envíen sus respuestas para poder combinar y ordenar la lista final de números primos. Si un trabajador sufre un retraso (por ejemplo, por tener un rango numérico superior con mayor costo de procesamiento), genera un cuello de botella (straggler problem) donde los recursos en otros núcleos quedan ociosos a la espera de completar la sincronización.

4. ¿Qué ocurre cuando se utilizan más trabajadores que núcleos disponibles en el equipo?

Ocurre una sobrecarga por la gestión de procesos. En lugar de ejecutar todas las tareas de forma verdaderamente paralela en hardware, la máquina virtual de Elixir/Erlang (BEAM) utiliza su scheduler para alternar la ejecución de los trabajadores mediante la multiplexación por división de tiempo (conurrencia). Esto introduce overhead por cambio de contexto (context switching), lo que puede congelar el rendimiento o incrementar el tiempo total.

5. ¿El tiempo de ejecución disminuye siempre al aumentar el número de trabajadores? Explique sus resultados.

No, no disminuye indefinidamente. Inicialmente, pasar de 1 a 2 o 4 trabajadores reduce el tiempo gracias al paralelismo en hardware. Sin embargo, al seguir aumentando el número de trabajadores, se alcanza un punto donde el costo de crear procesos, serializar y pasar mensajes, y combinar/ordenar los resultados finales en el coordinador supera la ganancia de velocidad. Además, por la Ley de Amdahl, la parte secuencial de la aplicación limita la aceleración máxima posible.

6. ¿Qué costos adicionales introduce la creación y comunicación entre procesos?

Memoria y tiempo de creación: Asignación de memoria inicial para los stacks y estructuras internas de cada proceso de la BEAM.

Sobrecarga de mensajes (Overhead): Copiar los datos entre las áreas de memoria separadas de los distintos procesos (no comparten memoria directamente). Costos de sincronización y ordenamiento: El trabajo adicional del coordinador para recibir, deserializar, concatenar y ordenar las sublistas enviadas por cada trabajador.

7. *¿Existe alguna diferencia entre ejecutar la solución con un solo proceso y ejecutarla con múltiples procesos?*

Sí, un solo proceso es secuencial, no realiza comunicación por mensajes para dividir tareas, no tiene sobrecarga de cambio de contexto ni sincronización, pero está limitado estrictamente al rendimiento de un único núcleo de CPU. Múltiples procesos, permite aprovechar múltiples núcleos procesando partes del intervalo al mismo tiempo. A cambio, requiere lógica de coordinación, gestión de mensajes y concatenación de resultados.

8. *¿La implementación realizada puede considerarse un sistema distribuido? Justifique su respuesta considerando la diferencia entre concurrencia, paralelismo y distribución.*

No estrictamente en la configuración básica descrita, sino como una aplicación concurrente y paralela en memoria compartida o máquina local, aunque la plataforma Elixir esté preparada para la distribución. Concurrencia es la habilidad de manejar múltiples tareas al mismo tiempo intercalando su ejecución (soporte lógico). Paralelismo es la ejecución física simultánea de múltiples tareas en diferentes núcleos del procesador al mismo tiempo (hardware local). Distribución es la ejecución de procesos a través de múltiples nodos físicos autónomos que no comparten memoria ni reloj global, conectados por una red.

Dado que el programa se ejecuta sobre una sola máquina virtual dentro del mismo nodo/equipo, aprovecha la concurrencia y el paralelismo. Solo se consideraría un sistema distribuido si los procesos trabajadores fueran desplegados explícitamente en diferentes máquinas de una red comunicándose mediante Distributed Erlang o sockets.